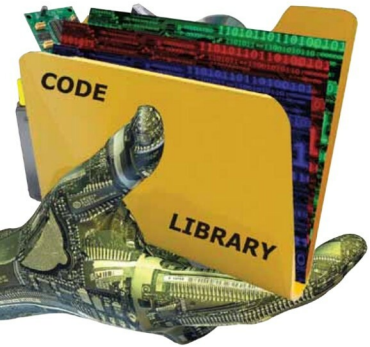

Let's Hook a Library Function

If you are a developer, and want to change the way a library function works, this article will give you a basic idea of how to get started—just enough knowledge to be able to experiment with your library functions. All code here is in C, and tested with GCC on Linux.

According to Wikipedia, "In computer programming, the term *hooking* covers a range of techniques used to alter or augment the behaviour of an operating system, applications, or other software components, by intercepting function calls or messages or events passed between software components. Code that handles such intercepted function calls, events or messages is called a 'hook'."

Intercepting a library call, and calling your own wrapper code, is also called Function Interposition.

Hooking has two benefits:

- You don't have to search for the function definition in the library, such as *libc (glibc* is the GNU C Library, and *libc* is almost half of the size of *glibc*) and change it. Seriously, this is a very nasty technical task (at least for me!).
- You don't need to recompile the library's source code.

Library functions and system calls

Please look at Figures 1 and 2 for a graphical representation of what happens when a library function is hooked.

Note: Non-trivial source code files shown in this article are available in a zip archive on the *LFY* website at http://linuxforu.com/article_source_code/libfunc_source.zip

Now let's look at hooking a library function. The simple *prog1.c* program below just allocates 10 bytes of memory from the heap, and frees it:

```
#include<stdio.h>
#include<malloc.h>
#include<stdlib.h>

int main(void)

{

int *p;

printf("calling from main...\n");

p=(int *)malloc(10);

if(!p)

        {

        printf("Got allocation error...\n");

        exit(1);

        }

printf("returning to main...\n");
```